

Manual: Ring Milling Application

Daan Kerkhof

Updated: September 2026

Contents

1 Overview	3
2 Installation	3
2.1 Installing the Requirements	3
2.2 Python Version	4
2.3 Running the Application	4
3 Configuration	4
3.1 Calibration File	5
4 Running the Application	5
5 Features	6
5.1 Live Camera View	6
5.2 Create Calibration	6
5.3 Load Calibration	7
5.4 Tool Path Selection	7
5.5 Removing Points	8
5.6 Clearing Points	8
5.7 Reload Points	8
5.8 Drilling Depth	8
5.9 G-code Generation	8
5.10 Save Flag	9
5.11 Information Panel	9
5.12 Application Log	9

6	Python Structure	10
6.1	Main Window	10
6.2	Calibration Window	10
6.3	Tool Path Window	10
6.4	Configuration Window	11
6.5	Core Components	11
7	Troubleshooting	11
7.1	No Calibration File Loaded	11
7.2	Invalid Calibration File	11
7.3	No Points Selected	11
7.4	Camera Problems	12
8	Safety and Operation	12

1 Overview

The Ring Milling Application is a graphical user interface (GUI) designed to select milling locations on a live microscope image and convert these locations into G-code for a milling machine.

The application uses a camera to provide a live image of the sample. The user can select points on this image corresponding to the desired milling locations. The selected pixel coordinates are converted into physical coordinates using calibration data. The application then generates G-code which can be used by the milling machine.

The main application window contains the live camera image, an information panel, a log window, and several control buttons. The available functions are:

- Select Tool Path
- Create Calibration
- Load Calibration
- Change Save Flag
- Show Configuration
- Manual

A calibration file must be loaded before a tool path can be selected. This prevents milling coordinates from being generated without the required calibration parameters.

“latex

2 Installation

The Ring Milling Application is written in Python. The application uses PyQt6 for the graphical user interface, OpenCV for camera image acquisition and processing, and NumPy for numerical operations.

The application requires the following Python packages and versions:

```
numpy==2.4.1
opencv-contrib-python==4.13.0.90
PyQt6==6.10.2
PyQt6-Qt6==6.10.1
PyQt6_sip==13.11.0
```

Using the specified versions is recommended to ensure that the application runs with the same dependencies as the tested version.

2.1 Installing the Requirements

It is recommended to use a Python virtual environment for the application. After creating and activating the virtual environment, install the required packages using:

```
pip install numpy==2.4.1
pip install opencv-contrib-python==4.13.0.90
```

```
pip install PyQt6==6.10.2
pip install PyQt6-Qt6==6.10.1
pip install PyQt6_sip==13.11.0
```

2.2 Python Version

The Python version used for this application was Python 3.12.3.

2.3 Running the Application

The application should be run from the project directory so that the internal Python modules, configuration files, and resources can be found correctly. Running `main.py` starts the application.

3 Configuration

The application uses a configuration file to store settings such as the camera index, save directory, and location of previously selected points.

The configuration can be inspected from the main application using the *Show Configuration* button.

The configuration window reads the configuration JSON file and displays each key and its corresponding value. These values cannot be changed within the application. The configuration file can be located in the resources folder: `.../resources/config.json` and can be edited via any text editor.

The configuration file determines:

- camera index
- directory for the generated code
- directory for previous points
- standard save flag
- rotate index

The `camera_index` parameter specifies which camera is used by the application. When cameras are accessed using libraries such as OpenCV, they are generally identified by an integer index. The index 0 normally refers to the first available camera, 1 to the second, and so on.

The `save_code_directory` parameter specifies the directory in which generated code or other application output is stored.

In the example configuration, the directory is:

```
/.../exports/g_archive
```

The `previous_points_directory` parameter specifies the location of the JSON file containing previously stored points:

```
/.../resources/previous_points.json
```

The stored points can be loaded by the application to restore or reuse points from a previous operation.

The `standard.save_flag` parameter controls whether the application's standard saving behaviour is enabled. It is a Boolean value and can therefore be either `true` or `false`. If set to `true`, all generated G-code files created will be saved in the `save_code_directory`.

The `rotate_index` parameter specifies the rotation mode used by the application. The value is interpreted by the application code and determines how captured or processed images are rotated.

3.1 Calibration File

Calibration information is stored in a JSON file. The required values are:

```
{
  "pixels_per_mm": ...,
  "camera_X_offset": ...,
  "camera_Y_offset": ...,
  "end_mill_gap": ...,
  "microscope_height": ...
}
```

These values represent:

- `pixels_per_mm`: conversion between image pixels and physical distance.
- `camera_X_offset`: fixed X offset between the camera and milling tool.
- `camera_Y_offset`: fixed Y offset between the camera and milling tool.
- `end_mill_gap`: distance between the end mill and the relevant reference position.
- `microscope_height`: microscope height used during calibration.

The application checks that all five values are present and that they are stored as numerical values before accepting a calibration file. Calibration files can be created via the application or via any text editor (given that the `.json` file has the proper structure).

4 Running the Application

After starting the application, by running `main.py`, the main window opens and the live camera feed is displayed.

The application starts the camera in a separate camera thread and continuously updates the displayed image.

Initially, the information panel indicates that a calibration file needs to be loaded.

The recommended workflow is:

1. Start the application.
2. Check the camera image.

3. Create a new calibration or load an existing calibration file.
4. Verify the calibration information.
5. Open the Tool Path Selection window.
6. Select the desired milling points.
7. Set the drilling depth.
8. Save the tool path.
9. Use the generated G-code with the milling system.

5 Features

5.1 Live Camera View

The main window displays the live image from the camera. The camera is accessed using OpenCV and the frames are transferred to the GUI through a separate camera thread.

The latest camera frame is stored by the main window and is used when opening the calibration or tool-path selection windows.

5.2 Create Calibration

The *Create Calibration* button opens the calibration window.

The calibration window displays a captured camera image and provides several input fields:

- Microscope Height
- End Mill Height
- Millimeter Length
- Coordinate 1
- Coordinate 2
- X Offset
- Y Offset

These values have to be measured before every sampling session. Because the distance between the sample and the CNC machine is important. And this file converts the pixels on the screen into real life distance. Here is explained what all the parameters are and how they can be determined:

The microscope height is the distance between the sample and the microscope in millimeters. This can be measured with a ruler.

The end mill height is the distance between the tip of the milling bit and the sample in millimeters. This value is especially important since overestimating it can cause the milling machine to mill too deep. This value can be measured with a ruler.

Millimeter length is how much millimeters are there between the two selected coordinates. Before opening the calibration window a ruler or something of known length should be placed in the image. Then, two points should be selected with *millimeter length* distance. For example, if millimeter length is

set to 10, the two points clicked on the screen should be 10 mm apart from each other. The two points can be selected directly on the camera image. Coordinate 1 is displayed in red and Coordinate 2 is displayed in blue.

Then the offset of the camera is needed. This value is fixed for the CNC machine since the camera is mounted at a fixed place. For the Two Trees TTC450 CNC machine, which is the prototype for this application the offset is the following:

```
camera_X_offset = -66.15  
camera_Y_offset = 8.25
```

Those were all the inputs. The output parameters are then calculated and displayed. After all required calibration values have been entered, the calibration can be saved as a JSON file using the *Save Calibration* button.

5.3 Load Calibration

The *Load Calibration* button opens a file-selection dialog where an existing JSON calibration file can be selected.

The application verifies that the following keys are present:

```
pixels_per_mm  
camera_X_offset  
camera_Y_offset  
end_mill_gap  
microscope_height
```

If the file is valid, the values are loaded into the application and the calibration status is enabled.

The loaded values are displayed in the information panel.

5.4 Tool Path Selection

The *Select Tool Path* button opens the Tool Path Selection window.

A calibration must be loaded before this window can be opened.

The Tool Path Selection window displays a captured camera image. Milling locations can be selected by clicking on the image.

Every selected location is displayed as a red circle with a cross.

The selected points are stored as image coordinates:

$$(x, y)$$

The first selected point is also used to determine the offset between the image center and the first milling point.

5.5 Removing Points

The *Remove last point* button removes the most recently selected point.

This can be used to correct an accidental mouse click without removing the other selected points.

5.6 Clearing Points

The *Clear Points* button removes all currently selected points.

The first-point calibration reference is also reset.

This allows a completely new tool path to be created.

5.7 Reload Points

The *Reload Points* button loads the previously saved set of points.

A warning is displayed before loading the previous points because the current tool path will be replaced.

The location of the previous-points file is determined by the configuration file.

5.8 Drilling Depth

The drilling depth can be changed using the drilling-depth input field.

The default drilling depth is:

1 mm

The input accepts values between 0 and 15 mm, with a resolution of 0.001 mm.

The selected value is passed to the G-code conversion function when the tool path is saved.

5.9 G-code Generation

When the *Save* button is pressed, the selected image coordinates are converted into G-code.

The conversion uses:

- selected image coordinates,
- X camera offset,
- Y camera offset,
- mill gap,
- first-point offset,
- pixels-per-mm calibration,

- drilling depth.

The resulting G-code is generated by the `coords_to_gcode` function.

A G-code export is written to the application's export location in `../exports/g_export.txt`. This file should be read by the G-code sender. If the saveflag is enabled, an additional timestamped copy is stored in the configured save directory.

5.10 Save Flag

The *Change Save Flag* button switches the additional G-code archive saving on or off.

When the save flag is enabled, the application saves a timestamped file using the current date and time.

The filename follows the format:

```
coordinates_YYYY-MM-DD_HH-MM-SS.txt
```

The save flag can only be changed after a calibration has been loaded.

5.11 Information Panel

After a calibration is loaded, the information panel displays the active calibration values.

The displayed information includes:

- X-offset
- Y-offset
- Mill Gap
- Camera Height
- Pixels per mm
- Save flag

This provides a quick way to verify which calibration and saving settings are currently active.

5.12 Application Log

The main window contains a log area which records important application events.

Each log entry contains the current time followed by a description of the event. Examples include:

```
[12:34:56] Application Started Successfully
[12:35:10] Loading Calibration
[12:35:12] Calibration Loaded Successfully
[12:36:01] Opening Tool Path Selection...
```

The log can be used to determine which operations have already been performed and to identify errors during normal operation.

6 Python Structure

The application is divided into GUI and core modules.

The main GUI components are:

- MainWindow
- CalibrationWindow
- ToolPathWindow
- ShowConfigWindow

6.1 Main Window

The MainWindow class controls the primary application interface.

Its responsibilities include:

- displaying the live camera image,
- starting and stopping the camera thread,
- loading calibration files,
- opening the calibration window,
- opening the tool-path window,
- displaying configuration information,
- changing the save flag,
- displaying application logs.

6.2 Calibration Window

The CalibrationWindow class is responsible for creating calibration files.

It receives a camera frame, allows the user to select two reference points, calculates the pixel distance, and determines the pixels-per-mm conversion.

The calibration information is finally stored as a JSON file.

6.3 Tool Path Window

The ToolPathWindow class handles the selection of milling points.

The user selects points by clicking on the camera image. The coordinates are stored in a list and displayed graphically on the image.

When the user saves the tool path, the coordinates are passed to the G-code conversion function.

6.4 Configuration Window

The `ShowConfigWindow` class displays the contents of the application's configuration JSON file.

Each configuration key and its corresponding value are displayed in the window.

6.5 Core Components

The GUI uses several core modules:

- `core.camera_thread`: handles camera acquisition.
- `core.read_config`: reads configuration settings.
- `core.g_code`: converts selected coordinates into G-code.

The camera is handled in a separate thread so that continuous image acquisition does not block the graphical user interface.

7 Troubleshooting

7.1 No Calibration File Loaded

If *Select Tool Path* is pressed without a loaded calibration, the application displays an error.

Load an existing calibration file using *Load Calibration*, or create a new calibration using *Create Calibration*.

7.2 Invalid Calibration File

A calibration file must contain all required numerical values.

Check that the JSON file contains:

```
pixels_per_mm  
camera_X_offset  
camera_Y_offset  
end_mill_gap  
microscope_height
```

7.3 No Points Selected

The tool path cannot be saved when no points have been selected.

Select at least one point on the camera image before pressing *Save*.

7.4 Camera Problems

If no camera image is displayed, check the camera connection and the camera index specified in the configuration.

Restarting the application after changing the camera configuration may be necessary.

8 Safety and Operation

The application generates G-code based on coordinates selected by the user. The generated G-code should therefore be checked before it is used with the milling machine.

In particular, verify:

- the calibration values,
- the camera offsets,
- the mill gap,
- the selected milling points,
- the drilling depth,
- the generated G-code.

The application should only be used when the camera calibration and milling setup have been verified.